METAVORIA

Société de services et d'ingénierie informatique — Tunisie

CAHIER DES CHARGES

Conception et développement d'une application web de

GESTION INTELLIGENTE DE TICKETS SUPPORT ASSISTÉE PAR IA

Projet de Fin d'Études (PFE)

Cycle Ingénieur — Filière : Génie Informatique

Étudiant(e) :	Fedi Zaghdoud
Encadrant entreprise :	Mohamed Amine Laribi
Encadrant académique :	[Nom de l'enseignant encadrant]
Durée du stage :	6 mois
Période :	Mars 2026 – Août 2026
Année universitaire :	2025 / 2026

Sommaire

- 1. Présentation générale
- 2. Contexte et problématique
- 3. Objectifs du projet
- 4. Périmètre du projet
- 5. Acteurs et utilisateurs
- 6. Besoins fonctionnels
- 7. Besoins non fonctionnels
- 8. Architecture et choix technologiques
- 9. Modélisation préliminaire (cas d'utilisation)
- 10. Contraintes du projet
- 11. Livrables attendus
- 12. Planning prévisionnel
- 13. Critères d'évaluation et de réussite
- 14. Glossaire

1. Présentation générale

1.1 Présentation de l'entreprise

Metavoria est une société tunisienne spécialisée dans les services et le conseil en informatique (développement web, intégration de solutions logicielles, infogérance, etc.). Dans le cadre de sa politique d'amélioration continue de ses processus internes, notamment la gestion du support technique, l'entreprise souhaite doter son équipe IT d'un outil moderne et intelligent de gestion des tickets.

1.2 Présentation du projet

Le présent document constitue le cahier des charges du Projet de Fin d'Études (PFE) intitulé « Smart Ticketing & Support Assistant » : une application web de gestion de tickets de support technique, enrichie de fonctionnalités d'intelligence artificielle permettant la classification automatique, la priorisation et la suggestion de solutions aux tickets, en s'appuyant sur l'historique des tickets déjà traités par l'entreprise.

2. Contexte et problématique

Actuellement, la gestion des demandes de support (incidents techniques, pannes, demandes d'accès, bugs applicatifs, etc.) au sein de l'entreprise se fait de manière peu structurée (échanges par e-mail, messagerie instantanée ou appels téléphoniques). Cette approche présente plusieurs limites :

- Absence de traçabilité centralisée des demandes et de leur historique de traitement.
- Temps de réponse et de résolution difficiles à mesurer et à améliorer.
- Répartition manuelle et peu optimisée des tickets entre les techniciens.
- Perte de connaissance : les solutions apportées à des problèmes récurrents ne sont pas capitalisées.
- Absence d'indicateurs de performance (KPI) pour le pilotage de l'activité support.

Le projet vise à répondre à ces limites en proposant une plateforme centralisée, capable non seulement de gérer le cycle de vie des tickets, mais également d'assister les équipes techniques grâce à des mécanismes d'intelligence artificielle exploitant les données historiques de l'entreprise.

3. Objectifs du projet

3.1 Objectif général

Concevoir, développer et déployer une application web permettant la gestion complète des tickets de support technique, intégrant un assistant intelligent basé sur l'IA (classification automatique, priorisation, suggestions de résolution).

3.2 Objectifs spécifiques

1. Digitaliser et centraliser le processus de création, suivi et clôture des tickets de support.
2. Mettre en place une classification automatique des tickets par catégorie (réseau, matériel, logiciel, accès, autre) à l'aide d'un modèle de traitement du langage naturel (NLP).
3. Mettre en place un système de priorisation automatique des tickets selon leur urgence et leur impact.
4. Développer un moteur de suggestion de solutions basé sur la recherche de similarité avec les tickets historiques déjà résolus (approche RAG – Retrieval Augmented Generation).
5. Fournir un tableau de bord analytique avec des indicateurs de performance (temps moyen de résolution, charge par technicien, tickets par catégorie, etc.).

6. Permettre une intégration avec les outils de communication interne existants (Slack, Microsoft Teams) pour la notification et la création rapide de tickets.
7. Assurer une gestion des rôles et des droits d'accès (utilisateur, technicien, administrateur).

4. Périmètre du projet

Le périmètre du projet couvre le développement d'une application web complète (frontend + backend + base de données) ainsi que l'intégration d'un module d'intelligence artificielle. Sont inclus dans le périmètre :

- La gestion du cycle de vie complet des tickets (création, affectation, traitement, clôture).
- Le module d'IA de classification, priorisation et suggestion de solutions.
- Le tableau de bord et les rapports statistiques.
- La gestion des utilisateurs, rôles et permissions.
- Le déploiement sur un environnement de test (conteneurisation Docker).

Sont explicitement exclus du périmètre (sauf accord ultérieur avec l'encadrant) :

- L'intégration avec un ERP ou CRM externe autre que ceux mentionnés.
- Le développement d'une application mobile native (une interface web responsive est suffisante).
- La mise en production définitive sur l'infrastructure de l'entreprise.

5. Acteurs et utilisateurs

Acteur	Description	Principales actions
Employé / Demandeur	Tout collaborateur de l'entreprise pouvant soumettre une demande de support.	Créer un ticket, suivre son statut, échanger avec le technicien, évaluer la résolution.
Technicien support	Membre de l'équipe IT chargé de traiter les tickets.	Consulter les tickets assignés, mettre à jour le statut, consulter les suggestions IA, résoudre le ticket.
Administrateur	Responsable de la gestion de la plateforme.	Gérer les utilisateurs et les rôles, configurer les catégories, consulter le tableau de bord global, gérer les paramètres du module IA.
Assistant IA (acteur système)	Composant automatisé du système.	Classifier les tickets, évaluer la priorité, proposer des solutions similaires issues de l'historique.

6. Besoins fonctionnels

6.1 Module Authentification et gestion des utilisateurs

- Inscription / connexion sécurisée (JWT) avec gestion des rôles (Employé, Technicien, Administrateur).
- Réinitialisation de mot de passe par e-mail.
- Gestion des profils utilisateurs (informations, service, poste).

6.2 Module Gestion des tickets

- Création d'un ticket avec titre, description, pièces jointes, catégorie (facultative, suggérée automatiquement si non renseignée).
- Affectation automatique ou manuelle du ticket à un technicien selon la charge de travail et la catégorie.
- Suivi du statut du ticket (Nouveau, En cours, En attente, Résolu, Clôturé).
- Fil de discussion (commentaires) entre le demandeur et le technicien.
- Notifications par e-mail et/ou en application lors des changements de statut.
- Évaluation de la satisfaction par le demandeur à la clôture du ticket.

6.3 Module Intelligence Artificielle

- Classification automatique de la catégorie du ticket à partir du texte de la description (NLP, modèle pré-entraîné ou API LLM).
- Calcul automatique d'un score de priorité (faible, moyenne, haute, critique) basé sur des mots-clés, l'historique et la catégorie.
- Recherche de tickets similaires déjà résolus dans l'historique (recherche vectorielle / embeddings) et suggestion de la solution associée au technicien.
- Génération d'un résumé automatique pour les tickets longs ou les fils de discussion étendus.
- Tableau de confiance : affichage du niveau de confiance de la suggestion IA au technicien (validation humaine obligatoire avant clôture).

6.4 Module Tableau de bord et reporting

- Indicateurs clés : nombre de tickets ouverts/fermés, temps moyen de résolution, taux de satisfaction, répartition par catégorie et par priorité.
- Filtres par période, par technicien, par service demandeur.
- Export des rapports (PDF / Excel).

6.5 Module Administration

- Gestion des catégories, priorités et règles d'affectation.
- Gestion des comptes utilisateurs et des rôles.
- Paramétrage du module IA (seuils de confiance, réentraînement / mise à jour de la base de connaissances).
- Intégration optionnelle avec Slack/Teams pour la création et la notification de tickets.

7. Besoins non fonctionnels

Catégorie	Exigence
Performance	Temps de réponse de l'interface inférieur à 2 secondes ; réponse du module IA inférieure à 5 secondes.
Sécurité	Authentification sécurisée (JWT), chiffrement des mots de passe (bcrypt), contrôle d'accès basé sur les rôles (RBAC), protection contre les failles courantes (XSS, injection SQL, CSRF).
Ergonomie / UX	Interface responsive, intuitive, accessible sur ordinateur et mobile.
Disponibilité	Application déployée via conteneurs Docker, disponible en environnement de test avec objectif de disponibilité de 99% durant les démonstrations.

Catégorie	Exigence
Scalabilité	Architecture permettant l'ajout futur de nouveaux modules ou l'augmentation du volume de tickets sans refonte majeure.
Maintenabilité	Code structuré, documenté, respectant les bonnes pratiques (linting, tests unitaires, architecture en couches).
Compatibilité	Compatible avec les navigateurs modernes (Chrome, Firefox, Edge).
Traçabilité	Journalisation (logs) des actions critiques : création, affectation, modification de statut des tickets.

8. Architecture et choix technologiques (proposition indicative)

Les technologies suivantes sont proposées à titre indicatif ; le choix final sera arrêté avec l'encadrant en fonction du niveau et des affinités techniques de l'étudiant(e).

Couche	Technologies proposées
Frontend	React.js (ou Angular/Vue) + TypeScript, TailwindCSS, gestion d'état (Redux/Zustand)
Backend	Node.js (NestJS/Express) ou Python (Django/FastAPI)
Base de données	PostgreSQL ou MySQL (données relationnelles) + base vectorielle (ex. pgvector, Pinecone, Chroma) pour la recherche par similarité
Module IA	API LLM (ex. Claude, OpenAI) ou modèle NLP open-source (ex. spaCy, HuggingFace Transformers) pour la classification et les embeddings
Authentification	JWT + bcrypt
Intégrations	API Slack / Microsoft Teams (webhooks), service d'envoi d'e-mails (SMTP / SendGrid)
Conteneurisation & CI/CD	Docker, Docker Compose, GitHub Actions / GitLab CI
Outils annexes	Git/GitHub pour le versioning, Postman pour les tests API, Swagger pour la documentation API

8.1 Architecture générale envisagée

Architecture en couches de type client-serveur : une application frontend (SPA) communiquant via une API REST (ou GraphQL) sécurisée avec un backend exposant les services métier, lui-même connecté à la base de données relationnelle et au module IA (interne ou via API externe). Un schéma d'architecture détaillé sera produit par l'étudiant(e) lors de la phase de conception.

9. Modélisation préliminaire (cas d'utilisation)

Les principaux cas d'utilisation identifiés à ce stade, à affiner lors de la phase d'analyse et de conception (diagrammes UML : cas d'utilisation, séquence, classes) :

- S'authentifier / Gérer son profil
- Créer un ticket
- Consulter / filtrer la liste des tickets
- Affecter un ticket à un technicien

- Mettre à jour le statut d'un ticket
- Consulter les suggestions de l'assistant IA
- Commenter / échanger sur un ticket
- Clôturer un ticket et évaluer la résolution
- Consulter le tableau de bord et les statistiques
- Administrer les utilisateurs, rôles et catégories

Il est demandé à l'étudiant(e) de produire, en phase de conception, un diagramme de cas d'utilisation global, des diagrammes de séquence pour les scénarios critiques (création de ticket avec classification IA, suggestion de résolution), ainsi qu'un diagramme de classes ou un modèle conceptuel de données (MCD).

10. Contraintes du projet

- Durée du stage : 6 mois, du mois de mars 2026 au mois d'août 2026, à réaliser dans les locaux de Metavoria et/ou en mode hybride selon la politique en vigueur.
- Respect des normes de codage et de l'architecture logicielle validées par l'encadrant.
- Utilisation prioritaire d'outils et technologies open-source ou disposant d'une offre gratuite/étudiante.
- Confidentialité des données utilisées pour les tests (anonymisation des données réelles de l'entreprise si utilisées).
- Point d'avancement hebdomadaire avec l'encadrant entreprise et suivi périodique avec l'encadrant académique.

11. Livrables attendus

8. Rapport de PFE conforme aux exigences de l'établissement académique.
9. Code source complet de l'application, versionné sur un dépôt Git.
10. Documentation technique (architecture, installation, guide d'utilisation, documentation de l'API).
11. Application déployée sur un environnement de démonstration (Docker).
12. Support de présentation (soutenance) et démonstration fonctionnelle du produit.
13. Jeu de données de test / scénarios de test.

12. Planning prévisionnel indicatif

Le stage se déroule sur 6 mois, de mars 2026 à août 2026. Le planning ci-dessous répartit ce volume sur l'ensemble de la période.

Phase	Contenu	Période	Durée
Phase 1 – Cadrage	Étude de l'existant, analyse des besoins, validation du cahier des charges, étude technologique	Mars 2026 (S1–S3)	3 semaines
Phase 2 – Conception	Modélisation UML, conception de la base de données, architecture technique, maquettes UI/UX	Mars – Avril 2026 (S4–S6)	3 semaines
Phase 3 – Développement Backend	API REST, authentification, gestion des tickets, base de données	Avril – Mai 2026 (S7–S11)	5 semaines
Phase 4 – Développement	Interfaces utilisateur, tableau de bord, intégration	Mai – Juin 2026 (S12–S16)	5

Phase	Contenu	Période	Durée
Frontend	avec l'API		semaines
Phase 5 – Module IA	Classification automatique, priorisation, moteur de suggestion (recherche vectorielle)	Juin – Juillet 2026 (S17–S21)	5 semaines
Phase 6 – Intégration & tests	Tests unitaires et fonctionnels, corrections de bugs, tests de performance	Juillet 2026 (S22–S24)	3 semaines
Phase 7 – Déploiement & documentation	Conteneurisation Docker, documentation technique, préparation de la soutenance	Août 2026 (S25–S26)	2 semaines
Phase 8 – Soutenance	Rédaction finale du rapport et présentation	Fin août 2026 (S27)	1 semaine

Ce planning (≈ 27 semaines réparties sur 6 mois, mars à août 2026) est donné à titre indicatif et pourra être ajusté avec Mohamed Amine Laribi, encadrant entreprise, en fonction du niveau d'avancement de Fedi Zaghdoud.

13. Critères d'évaluation et de réussite

- Conformité de l'application aux besoins fonctionnels et non fonctionnels décrits dans ce document.
- Qualité de l'architecture logicielle et respect des bonnes pratiques de développement.
- Pertinence et performance du module d'intelligence artificielle (précision de la classification, pertinence des suggestions).
- Qualité de l'interface utilisateur et de l'expérience utilisateur.
- Qualité de la documentation technique et du rapport de PFE.
- Capacité de l'étudiant(e) à justifier ses choix techniques et à présenter son travail lors de la soutenance.

14. Glossaire

Terme	Définition
PFE	Projet de Fin d'Études
NLP	Natural Language Processing — traitement automatique du langage naturel
RAG	Retrieval Augmented Generation — génération de réponses enrichie par une recherche documentaire
JWT	JSON Web Token — standard de jeton d'authentification
RBAC	Role-Based Access Control — contrôle d'accès basé sur les rôles
KPI	Key Performance Indicator — indicateur clé de performance
MCD	Modèle Conceptuel de Données

Document rédigé à titre de proposition initiale. Il pourra être amendé conjointement par l'étudiant(e), l'encadrant entreprise et l'encadrant académique lors de la phase de cadrage du projet.